Product Recommendation Backend

Step-by-Step Detailed Explanation

Django · scikit-learn · TF-IDF · Cosine Similarity · REST API
Content-Based Filtering with Multi-Signal User Profiling

Step 01 — Project Overview

This project is a **pure REST API backend** built with Django that serves personalised product recommendations to users. It uses **Content-Based Filtering** — a Machine Learning technique that recommends products similar to what a user has already liked, bought, or browsed — without needing data from other users.

What problem does it solve?

E-commerce platforms need to surface relevant products to each user from a large catalog. This system learns each user's taste from three signal types and ranks the entire product catalog accordingly, returning the top-N most relevant items per request.

Key capabilities:

- **Personalised recommendations** — tailored per user based on ratings, purchases, and browsing
- **Item-to-item similarity** — "customers who liked X also liked..." style suggestions
- **Cold-start fallback** — returns globally top-rated products for brand-new users
- **Signal ingestion** — REST endpoints to log ratings, purchases, and browsing events in real time
- **Model management** — retrain and inspect the ML model via API or CLI

Layer	Technology	Role
Client	Any HTTP client	Sends requests, receives JSON
API Layer	Django + DRF	Routing, validation, response shaping
ML Engine	scikit-learn	TF-IDF vectoriser + cosine similarity
Persistence	SQLite / PostgreSQL	Products, users, signals, logs
Model Store	Filesystem (.pkl)	Serialised trained recommender

Step 02 — Tech Stack

Library	Version	Why it is used
Django	4.x	Web framework — ORM, management commands, URL routing
Django REST Framework	3.x	Serialisers, APIView, response helpers
scikit-learn	1.x	TfidfVectorizer, cosine_similarity, MinMaxScaler
pandas	2.x	Building and manipulating the product corpus DataFrame
numpy	1.x	Fast array operations for similarity scoring
SQLite	built-in	Zero-config dev database (swap to PostgreSQL in prod)
pickle	stdlib	Serialising the trained model object to disk

Why Content-Based Filtering?

- **No cold-start for items** — works as soon as a product has text metadata
- **No cross-user data needed** — privacy-friendly; each user profile is independent
- **Interpretable** — you can explain why a product was recommended (shared tags / words)
- **Scalable training** — similarity matrix is precomputed once; serving is just a lookup

Step 03 — Project Structure

The project follows a standard Django layout with the ML code living inside the **recommender** app:

```
product_recommender/
├── settings.py              # Django configuration
├── urls.py                  # Top-level URL routing
├── manage.py                # Django CLI entry point
├── seed.py                  # Demo data loader
├── db.sqlite3               # Auto-created database
├── ml_models/
│   └── content_based.pkl    # Trained model (auto-created after training)
└── recommender/
    ├── models.py            # Database schema (7 models)
    ├── serializers.py       # DRF request/response shapes
    ├── views.py             # All API endpoint logic
    ├── ml_engine.py         # ContentBasedRecommender class
    └── management/
        └── commands/
            └── train_recommender.py   # python manage.py train_recommender
```

File Responsibilities at a Glance

File	Responsibility
models.py	Defines all DB tables and their relationships
ml_engine.py	Trains TF-IDF, computes similarity, scores & ranks products
views.py	Handles HTTP requests, calls ML engine, returns responses
serializers.py	Validates input data and shapes output JSON
urls.py	Maps URL patterns to view classes/functions
settings.py	Django config: installed apps, DB, DRF settings
seed.py	Inserts demo products, users and interactions for testing
train_recommender.py	Management command: build corpus → fit model → save .pkl

Step 04 — Database Schema (models.py)

There are **7 Django models**. They split into three groups: catalog, user signals, and logging.

Group 1 — Product Catalog

Category

Organises products into top-level buckets (Electronics, Books…)

Fields: id, name (unique), slug (unique)

Product

Core product record. Holds metadata used to build TF-IDF vectors.

Fields: id, name, description, category(FK), tags(JSON), price, avg_rating, rating_count, created_at

Group 2 — User Signal Tables

Three tables capture different types of user behaviour. Together they form the **user profile** that the recommender uses to personalise results.

Model	Signal Type	Key Fields	Used For
User	Identity	external_id	Links signals to an identity
ProductRating	Explicit	user, product, score(1-5), review_text	Strongest signal — user stated preference
PurchaseHistory	Implicit	user, product, quantity	Strong implicit signal — real money spent
BrowsingEvent	Implicit	user, product, event_type	Weak signal — attention / intent

BrowsingEvent.event_type choices:

Event Type	Meaning	Engine Weight
view	User saw the product in a listing	1
click	User opened the product page	2
add_to_cart	User added to shopping cart	4
wishlist	User saved to wishlist	3

Group 3 — Audit

RecommendationLog stores every recommendation event: which user, which products were returned, which strategy was used, and when. Useful for A/B testing and offline evaluation.

Step 05 — ML Engine (ml_engine.py)

The **ContentBasedRecommender** class is the heart of the system. It has three phases: corpus building, training, and inference.

Phase 1 — Building the Product Corpus

Each product is converted into a single enriched text string called its **corpus**. This text combines every piece of information available about the product:

```
# build_product_corpus() — what goes into each product's text

corpus_text = " ".join(filter(None, [
    p.name,           # e.g. "Sony WH-1000XM5 Headphones"
    p.description,    # "Premium wireless noise-cancelling headphones..."
    tags_text,        # "wireless noise-cancelling audio bluetooth"
    category_text,    # "Electronics"
    reviews_text,     # ALL review texts written about this product
]))
```

Including review texts is important — it adds real user language to the product's representation, making similarity more semantic and less keyword-dependent.

Phase 2 — Training (fit)

Training has two sub-steps:

2a. TF-IDF Vectorisation

TF-IDF (Term Frequency-Inverse Document Frequency) converts each product's text into a numeric vector of up to **5,000 dimensions**. Words that appear often in one product but rarely across all products get high scores — these are the discriminating keywords.

```
self.vectorizer = TfidfVectorizer(
    max_features = 5000,    # vocabulary cap
    stop_words   = "english",  # ignore "the", "is", "and" …
    ngram_range  = (1, 2),  # single words AND two-word phrases
    min_df       = 1,       # include terms that appear even once
)
self.product_matrix = self.vectorizer.fit_transform(corpus_df["corpus"])
# Shape: (n_products, vocab_size)
```

2b. Cosine Similarity Matrix

After vectorisation, cosine similarity is computed for **every product pair**. Cosine similarity measures the angle between two vectors — 1.0 means identical, 0.0 means completely unrelated. The result is an N×N matrix stored in memory.

```
self.similarity_matrix = cosine_similarity(self.product_matrix)
# Shape: (n_products, n_products)
# similarity_matrix[i][j] = how similar product i is to product j
# Value range: 0.0 (no overlap) to 1.0 (identical)
```

2c. Quality Score (avg_rating normalisation)

```
self.rating_scores = MinMaxScaler().fit_transform(
    corpus_df[["avg_rating"]]
).flatten()
# Scales all average ratings to [0, 1]
# Used as a quality multiplier in the final score
```

Phase 3 — Inference (get_recommendations)

When a recommendation request arrives, the engine runs these steps:

Step 1 — Build user profiles

Three dictionaries are built from DB queries — one each for purchases, browsing, and ratings. Each maps {product_id: signal_strength}.

Step 2 — Identify seed products

Products the user has explicitly rated are preferred as seeds. Fallback order: ratings → purchases → browsing events. Seeds drive the similarity lookup.

Step 3 — Aggregate similarity scores

For each seed product, retrieve its row from the similarity matrix. Multiply by the user's signal strength for that seed. Sum across all seeds to get one aggregate similarity score per candidate product.

Step 4 — Normalise

Divide the aggregate similarity vector by its maximum value, so all scores are in [0, 1].

Step 5 — Compute final score

Combine similarity and quality using the weighted formula below.

Step 6 — Filter & rank

Optionally exclude already-seen products. Sort by final score descending. Return top-N.

The Scoring Formula

```
final_score(p) = 0.6 * similarity_score(p) + 0.4 * quality_score(p)

Where:
  similarity_score(p) = normalised aggregate cosine similarity to user seed products
  quality_score(p)    = normalised global average rating of product p

Signal weights within similarity aggregation:
  explicit rating  × 3   (strongest — user stated their preference)
  purchase history × 2   (strong — real transaction)
  browsing event   × 1   (weak — attention only)
```

User Signal Weight Rationale

Signal	Weight	Reasoning

Explicit rating (1–5 stars)	3×	Most reliable — user consciously chose a score
Purchase history	2×	High intent — user spent real money
Wishlist / save	3 pts	Strong intent but no commitment yet
Add to cart	4 pts	Strongest browsing signal — near-purchase intent
Click (product page)	2 pts	Moderate interest
View (listing)	1 pt	Weakest — could be accidental

Step 06 — API Layer (views.py)

All HTTP handling lives in **views.py**. It uses Django REST Framework's **APIView** class for class-based views and **@api_view** for simple function views.

Singleton Model Loading

The trained recommender is loaded once and kept in memory as a module-level variable. This avoids re-deserialising the large .pkl file on every request:

```
_recommender = None

def get_recommender():
    global _recommender
    if _recommender is None or not _recommender._is_fitted:
        _recommender = ContentBasedRecommender.load()   # reads .pkl once
    return _recommender
```

View Groups

View Class / Function	Method	Endpoint	What it does
ProductListCreateView	GET	/api/products/	List all products
ProductListCreateView	POST	/api/products/	Create a product
ProductDetailView	GET	/api/products/<id>/	Get one product
ProductDetailView	PUT	/api/products/<id>/	Update product (partial)
ProductDetailView	DELETE	/api/products/<id>/	Delete product
SimilarProductsView	GET	/api/products/<id>/similar/	Item-to-item similarity
RatingCreateView	POST	/api/users/<uid>/ratings/	Log a rating
PurchaseCreateView	POST	/api/users/<uid>/purchases/	Log a purchase
BrowsingEventCreateView	POST	/api/users/<uid>/browse/	Log a browse event
UserRecommendationsView	GET	/api/users/<uid>/recommendations/	Get personalised recs
retrain_model	POST	/api/admin/retrain/	Retrain model in-process
model_status	GET	/api/admin/model-status/	Check model health

Recommendation Endpoint — Detailed Flow

1. Resolve user: get_or_create User row from external_id path param
2. Parse query params: top_n (default 10), exclude_seen (default true)
3. Load the singleton recommender; return 503 if model not trained
4. Query DB for the user's purchases, browsing events, and ratings

5. Call recommender.get_recommendations() — returns [{product_id, score}]
6. Bulk-fetch full Product objects for all returned IDs
7. Merge score + product data into enriched dicts
8. Write a RecommendationLog record for audit/analytics
9. Serialise and return JSON response

Step 07 — Serializers (serializers.py)

DRF serializers serve two purposes: **input validation** (ensure incoming JSON has the right fields and types) and **output shaping** (convert model instances to JSON).

Serializer	Model	Notable logic
CategorySerializer	Category	Read-only; nested inside ProductSerializer
ProductSerializer	Product	Exposes category as nested object on read; accepts category_id on w
ProductRatingSerializer	ProductRating	validate_score() enforces 1.0–5.0 range
PurchaseHistorySerializer	PurchaseHistory	Standard — no custom validation
BrowsingEventSerializer	BrowsingEvent	event_type is validated against TextChoices enum
RecommendationSerializer	(custom)	Not a ModelSerializer — combines score (float) + nested ProductSeri

Score Validation Example

```
class ProductRatingSerializer(serializers.ModelSerializer):
    def validate_score(self, value):
        if not (1.0 <= value <= 5.0):
            raise serializers.ValidationError("Score must be between 1.0 and 5.0.")
        return value
```

Step 08 — Setup & Run (Step-by-Step)

Follow these four steps to get the project running locally from scratch:

Step 01 — Install dependencies

```
$ pip install django djangorestframework scikit-learn pandas numpy
```

Installs all required Python packages into your environment.

Step 02 — Create database tables

```
$ python manage.py migrate --run-syncdb
```

Django reads models.py and creates the SQLite tables. --run-syncdb is used because there are no migration files included.

Step 03 — Seed demo data

```
$ python seed.py
```

Inserts 5 categories, 15 products, 3 users (alice/bob/carol), and their ratings/purchases/browsing events. Also computes initial avg_rating on each product.

Step 04 — Train the ML model

```
$ python manage.py train_recommender
```

Builds TF-IDF corpus from all products + reviews. Fits the vectoriser. Computes the full cosine similarity matrix. Saves to ml_models/content_based.pkl.

Step 05 — Start the dev server

```
$ python manage.py runserver
```

Starts Django on http://127.0.0.1:8000/. The API is now live.

What happens during training (detailed)

```
# Inside train_recommender management command:

products_qs = Product.objects.all()          # fetch all products
ratings_qs  = ProductRating.objects.all()    # fetch all reviews

corpus_df = ContentBasedRecommender.build_product_corpus(products_qs, ratings_qs)
# → DataFrame with columns: product_id, corpus, avg_rating, rating_count, category_id, ta
gs

recommender = ContentBasedRecommender()
recommender.fit(corpus_df)                   # TF-IDF + cosine sim + scaler
recommender.save("content_based")            # writes ml_models/content_based.pkl
```

Step 09 — Full API Reference

Products

Method	URL	Request Body	Response
GET	/api/products/	—	Array of product objects
POST	/api/products/	{name, description, category_id, tags, price}	Created product
GET	/api/products/<id>/	—	Single product
PUT	/api/products/<id>/	Any product fields (partial OK)	Updated product
DELETE	/api/products/<id>/	—	204 No Content
GET	/api/products/<id>/similar/?top_n=10	—	{product_id, similar:[…]}

User Signals

Method	URL	Request Body	Notes
POST	/api/users/<uid>/ratings/	{product, score, review_text}	score: 1.0–5.0; upserts if rating exists
POST	/api/users/<uid>/purchases/	{product, quantity}	Appends a new purchase record
POST	/api/users/<uid>/browse/	{product, event_type, session_id}	event_type: view/click/add_to_cart/wishlist

Recommendations

```
GET /api/users/<user_id>/recommendations/?top_n=10&exclude;_seen=true

# Response:
{
  "user_id": "user_alice",
  "recommendations": [
    {
      "product_id": 3,
      "score": 0.847,
      "product": {
        "id": 3, "name": "Bose QuietComfort 45",
        "category": {"id":1,"name":"Electronics","slug":"electronics"},
        "tags": ["wireless","noise-cancelling","audio"],
        "price": "329.00", "avg_rating": 0.0, "rating_count": 0, ...
      }
    },
    ...
  ]
}
```

Model Management

Method	URL	Description	Response
GET	/api/admin/model-status/	Check if model is trained	{is_fitted, product_count, vocab_size}
POST	/api/admin/retrain/	Retrain in-process	{detail, products, vocab_size}

Step 10 — Production Hardening Checklist

The project ships as a development-ready prototype. Before deploying to production, the following gaps must be addressed:

Area	Current State	Production Fix
Secret key	Hardcoded insecure string	Load from environment variable (python-decouple / os.environ)
Database	SQLite (single-file, no concurrency)	PostgreSQL via DATABASES setting
Authentication	None — all endpoints are open	Add JWT (djangorestframework-simplejwt) or session auth
Model storage	Local filesystem .pkl	Persistent volume (S3, GCS) shared across all instances
Retraining	Synchronous in-process (blocks request)	Celery task triggered on schedule (Celery Beat) or event
Caching	No caching	Redis cache for hot user recommendation results (cache_page or manual)
Scaling	Single-process Django dev server	Gunicorn + multiple workers behind Nginx
Monitoring	Python logging only	Structured logs + Sentry for error tracking
Similarity matrix	Entire N×N matrix in RAM	For >100K products, switch to approximate nearest-neighbour (FAISS, Ann

Recommended Retraining Strategy

Environment	Approach	How to trigger
Development	POST /api/admin/retrain/	Manual HTTP call during development
Staging	python manage.py train_recommender	Run after data migrations in CI/CD pipeline
Production	Celery Beat scheduled task	Nightly (or after N new interactions) via message queue

Scaling the ML Component

The current $O(N^2)$ similarity matrix works well up to ~10,000 products (uses ~800MB RAM). Beyond that, switch to **Approximate Nearest Neighbour** search:

```
# For large catalogs: replace cosine_similarity matrix with FAISS
import faiss

index = faiss.IndexFlatIP(vocab_size)      # inner-product = cosine on L2-normed vecs
index.add(normalised_product_matrix)       # add all products

# At query time:
distances, indices = index.search(query_vector, top_n)
# Returns top_n similar products in milliseconds even for millions of items
```

This project demonstrates end-to-end ML system design: data modelling, feature engineering with TF-IDF, multi-signal user profiling, a weighted scoring function, REST API design, and model persistence — all within a clean, well-structured Django codebase.